

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/370496703>

Software Defects Prediction using Machine Learning Algorithms

Conference Paper · May 2023

CITATIONS

0

READS

744

3 authors, including:



Madadi Vijayakamal

Jawaharlal Nehru Technological University, Hyderabad

12 PUBLICATIONS 2 CITATIONS

SEE PROFILE

Software Defects Prediction using Machine Learning Algorithms

Jyothi kethireddy¹, Dr. E. Aravind², Dr. M V Kamal³

¹ Research Scholar, Chailthanya Deemed to be University.

²Associate Professor, Dept. of CSE, Chailthanya Deemed to be University.

³Associate Professor, Dept. of CSE (DS), MRCET Autonomous Campus, Hyderabad.

¹kethireddy.jyothi@gmail.com, ²aravind@chaitanya.edu.in, ³kamalmv@gmail.com

Abstract. The most significant feature of a software is its quality. Software Defect Prediction has grown significantly in popularity over the past several years and may directly impact quality. Defective software modules significantly lower software quality, which results in cost overruns, missed deadlines, and far higher maintenance expenses. One of the best strategies in this direction is to predict software problems using machine learning (ML) methods. In this study, ML algorithms used in this inquiry include ANNs (artificial neural networks), RF (random forest), RT (random tree), DT (decision table), LR (linear regression), GP (gaussian processes), SMOreg, and M5P. For the purpose of predicting future software defects, a novel software defect prediction model is put forth. The defect prediction is grounded in the past. The outcomes demonstrated that a combination of ML techniques might be used to anticipate software problems in an accurate manner over other algorithms in terms of defect prediction accuracy.

Keywords: Software Quality, Defect prediction, Machine Learning, Artificial neural networks (ANNs), random forest (RF), random tree (RT), decision table (DT), linear regression (LR), gaussian processes (GP), SMOreg, and M5P

1 Introduction

The most crucial aspect of a software is its quality. It is the extent to which a software programme complies with both openly stated and implicit needs or expectations of a client. It has a significant effect on business since it has the power to enhance or degrade a company's brand image. If not handled correctly, it may result in budget overruns, schedule delays, and much higher maintenance expenses. If a problem arises after the programme has been made "live," that is, after the product has been sold, the entire problematic module must be reviewed for bugs again and the code must be changed as part of maintenance. Regression testing is then used to address the different levels.

Software classes or modules that are more likely to have bugs can easily be identified by software defect prediction models. Before the product is deployed, the maintenance costs are kept to a minimum. [1] The production of faultless products and the delivery of high-caliber software projects will result from the early discovery of the error. The software development life cycle must include software defect

prediction. According to the product's quality, the software's complexity and dependencies are growing. software defect prediction is a high priority task to improve the software product's quality.

Quality, according to the International Software Testing Qualifications Board (ISTQB), "The extent to which a part, system, or process complies with user/customer needs and expectations, or requirements ". Software quality is described as "the entirety of a software product's functionality and characteristics that have an impact on its capacity to meet stated or implied demands." [ISTQB V 2.2 Standard Glossary of Terms Used in Software Testing].

The software sector is expanding rapidly and technologically. Consequently, predicting software dependability is a crucial responsibility in the software development process [2]. A software bug is a flawed behaviour of the software system that results from an actual or potential security policy violation during the course of an application's execution. It is mostly brought by incorrect software system definition or inappropriate software system development.

In order to meet three crucial requirements, analysis and prediction of flaws are crucial, according to the reference by [3] It first supports the project manager in arranging the testing procedure and in evaluating the project's progress.

Second, it aids in examining the product's quality. Finally, it enhances the product's dependability and functionality. Different metrics that have been supplied by the prior fault prediction can be used to identify the fault-prone modules. To improve the effectiveness of the testing process, some of the key data is retrieved, including the quantity of faults, their location and severity, and their distribution.

Enhancing the entire testing process by focusing on fault-prone modules and identifying the refactoring candidates who are deemed to be most likely to have faults are the two key benefits of software fault prediction [4].

There are a number of reasons why the models used for software engineering costs and schedules, their estimate, etc., are implemented, including

- Budgeting: Although it is the primary impact, it is not the only goal. The system's "overall accuracy" is the most crucial element.
- Offering cost and timing estimates with regard to modules, stages, and processes is yet another crucial aspect of project planning and control.
- The capacity to focus on the project timing and expenses associated in the project decisions is a component of trade off and risk analysis (staffing, scoping, tools, reuse, etc.).
- Investment analysis for software improvement: This includes the extra costs and work needed for other techniques, such as recycling, tools, inventories, process maturity, etc.

Few engineers and academics have attempted to apply the results from one project to another in an effort to overcome such obstacles [5], [6] Cross-Project Defect Prediction is the phrase used to describe this method of building defect models utilising data from other projects (CPDP). It involves the process of integrating a prediction model created for one project into another.

Defect analysis and prediction in software programming may definitively identify probable faults in the software and aid in identifying the modules that are more susceptible to such issues. It can help the engineers focus their limited resources on

the software framework's components that are most vulnerable to problems. Building defect prediction models for a software framework is beneficial for many development or maintenance tasks, such as monitoring quality assurance and assessing software quality (QA).

In this study, the supervised machine learning algorithms are—the Artificial Neural Network (ANN), the Random Forest (RF), the Random Tree (RT), the Decision Table (DT), the Linear Regression (LR), the Gaussian Processes (GP), the SMOreg, and the M5P—will be analysed and evaluated. In addition to providing a comparative analysis of the chosen ML algorithms, the study demonstrates the performance accuracy and capabilities of ML algorithms in software bug prediction.

2. literature review

Software bug prediction using machine learning techniques has been the subject of several research. To anticipate the problematic modules, for instance, the study in [3] suggested using a linear auto-regression (AR) technique. Based on previous data of the program's accumulated defects, the study forecasts future software problems. The study also assessed and contrasted the AR model and employed the Root Mean Square Error (RMSE) metric with the Known Power Model (POWM). The study also employed three datasets for evaluation, and the outcomes were encouraging.

The investigations in [7], [8] examined how different ML techniques applied to fault prediction. The most significant prior studies on each ML approach as well as the most recent developments in machine learning-based software bug prediction were included in Sharma and Chandra's paper [7]. The results of this study can be utilised as a foundation or first step in future work on software bug prediction.

Five well-known machine learning (ML) techniques, including Artificial Neural Networks (ANNs), Particle Swarm Optimization (PSO), Decision Trees (DT), Naive Bayes (NB), and Linear Classifiers, were explored by [9](LC). The study revealed significant findings, such as the following: ANN has the lowest error rate, followed by DT, but the linear classifier outperforms other algorithms in terms of defect prediction accuracy; DT, BL, ANN, SVM, RBL, and EA are the most popular methods used in software defect prediction; and the following metrics are frequently used in software defect prediction studies: Line Of Code (LOC) metrics, object-oriented metrics like cohesion, coupling, and inheritance, as well as other metrics known as hybrid metrics that used both procedural and object-oriented metrics, were all found to be useful in software defect prediction studies. NASA datasets and PROMISE datasets were also frequently used.

Using machine learning, R. Malhotra published a thorough comprehensive assessment of software bug prediction approaches in [10] (ML). The paper reviewed all studies conducted between 1991 and 2013; it analysed machine learning (ML) techniques for software bug prediction models; it evaluated their performance; it compared ML to statistical techniques; it compared various ML techniques; and it summarised the strengths and weaknesses of the ML techniques.

The study established a benchmark in [11] so that various bug prediction algorithms could be easily and effectively compared. The study proposed a novel technique and assessed its performance by developing a strong comparison with

previous approaches using the provided benchmark. It also gave a thorough comparison between two well-known bug prediction approaches.

A model for an object-oriented Software Bug Prediction System was created by D. L. Gupta and K. Saxena [12]. (SBPS). Similar types of defect datasets from the Promise Software Engineering Repository were merged for the study. The performance metric was used in the study to assess the suggested model (accuracy). The study's findings also revealed an average recommended model accuracy of more than 60%.

Sofian et.al, [13] shown that the AUC is a useful statistic that can be used to identify problematic modules early in the software development process and to increase the precision of ML approaches. An application employing the evolutionary algorithm for fault proneness prediction was reported by Parashar et al. [14] The programme gets its values from an open source software project, including the object-oriented metrics and count metrics values. The application's values are used as inputs by the genetic algorithm to create rules that are used to divide software modules into faulty and non-defective modules. Utilize the genetic algorithm applet to finally view the results.

As an alternative to conventional techniques, author presented a software fault prediction using code profiles [15], [16]. This work use vectors to train the ML algorithm. By examining the usage of language features, the vectors are created from the parse trees of the source codes. As feature variables, the parse trees are employed. The code profiles show promise in replacing the conventional measures for forecasting software faults. However, both metrics-based and code profile-based algorithms have limited predictive ability when it comes to cross-version fault predictions.

Furthermore, the research in [17], [18], [19] included a variety of ML methods and gave an overview of how to use ML to forecast software defects. The research helped the developer improve the product's quality by using appropriate data mining techniques and effective software measurements. According to the study in [18], [19] the most relevant metrics for defect prediction were Response for class (ROC), Line of code (LOC), and Lack of Coding Quality (LOCQ).

In this study Eight machine learning methods will be used in this inquiry to offer a prediction model for software problems. The algorithms are random forest, random tree, decision table, linear regression, gaussian processes, SMOReg, and M5P, among others. Artificial neural networks (ANNs) are also included. This study tries to evaluate how well machine learning methods forecast software problems.

3. Methodology

For the comparative analysis investigation, a variety of algorithms were investigated. The selection was created in a way that it includes the most well-liked and often utilised machine learning algorithms. The chosen algorithms and a short description of each are listed below:

3.1 Artificial neural network

A mathematical and computational model called a neural network, sometimes known as a "Artificial Neural Network" (ANN), is based on biological neural networks. It is made up of an artificial network of neurons that are linked and work together to process information. During the learning phase, it is an adaptable system that modifies its structure in response to the information flow, whether internal or external. It resembles the biological brain system in imitation. An ANN's two primary parts are Nodes (processing units) and Links (connection between nodes). They are referred to as the system having connections between neurons that may communicate with one another by exchanging signals. The following describes how an ANN functions. First, the input layer nodes of the neural network accept the variable values as input. The connections that connect the nodes are given weights. These neural networks may learn and adapt because the numerical weights are adjusted based on experience. Nodes are transversed, and the network is used to determine the variable values. Each link's weight has an impact on the variable value result.

3.2 Random forest

A supervised classification and regression learning algorithm is the random forest (RF) algorithm. Instead of employing a single tree decision, it generates a forest of decision trees and uses committee voting to choose the optimal option.

RF combines the benefits of bagging and random selection, two machine learning techniques. By training each tree on a bootstrap sample of the training data, Bagging creates predictions based on a majority vote of the trees. At each node, random feature selection looks for the optimal split over a chosen subset of the characteristics. Good outcomes are produced by random characteristics and inputs. When building a tree, RF splits at each node using a randomly chosen subset of characteristics.

The primary traits of RF are that it performs more quickly than boosting and bagging. RF can withstand noise and outliers. Compared to currently known methods for categorization, RF is reportedly more accurate. For huge data sets, RF is effective. The RF technique is effective at guessing missing data.

3.3 Random Tree

Random Tree is a visual depiction of every solution conceivable under a given set of circumstances. We refer to it as a tree since, in essence, it contains a root and several branches that result in numerous data sets to form a decision tree.

Leo Breiman and Adele Cutler invented the concept of random trees. The algorithm can handle issues involving classification and regression. A group (ensemble) of tree predictors known as a forest includes random trees. The random trees classifier uses each tree in the forest to classify the input feature vector, and then it outputs the class label that earned the majority of "votes." In a regression scenario, the classifier response is the mean of all the responses across all the forest's trees.

Random Trees are simply the merging of two already-existing machine learning algorithms: single model trees and random forest principles. Model trees are decision trees with each leaf containing a linear model that is optimised for the local subspace the leaf is describing. Tree diversity is produced by two different methods of

randomization, and Random Forests have been found to significantly increase the performance of single decision trees.

3.4 Linear Regression

An algorithm for supervised learning is LR. By establishing a linear regression connection between an independent variable (x) and a dependent variable value (y), it forecasts the outcomes.

It is feasible to measure the projected effects and model them against a variety of input factors using the mathematical research technique known as linear regression. A linear link between dependent and independent variables is established using this technique for data assessment and modelling. Thus, from analysis and learning to the results of the most recent training, this technique would represent correlations between dependent variables and independent variables.

3.5 Gaussian Processes

Gaussian Processes are a distribution of potential outcomes' probability. It is a collection of random variables represented by time or space, where each group of those random variables has a multivariate distribution.

Hard machine learning issues are frequently solved using Gaussian process models. They are appealing due of their computational simplicity and adaptable non-parametric character. Very potent statistical techniques that provide accurate estimates of the uncertainty in our predictions and general model selection processes framed as nonlinear optimization problems may be applied when treated within a Bayesian framework. Generic sparse approximations have recently been introduced, addressing their fundamental issue of expensive computational scaling. Many of the deep notions used in the extensive mathematical literature on GPs are not necessary to effectively comprehend the majority of machine learning applications.

3.6 Decision table

Decision table is a structured approach for creating requirements from business rules. It's a classifier that can simulate intricate reasoning. Conditions are marked as true (T) or false (F) in a DT (F). A business logic rule that explains the particular set of cases is represented by each column in the table.

Decision tables, decision networks, and decision tables are categorization methods for forecasting. Machine learning algorithms produce these. A decision table is made up of a hierarchy of tables where each entry is split down into its component parts by the values of two more characteristics to create a new table. The arrangement resembles dimensional stacking.

3.7 SMOReg

SMOReg uses many techniques for parameter learning and provides a vector machine for regression. The method changes the nominal properties into binary ones and substitutes the missing values.

3.8 M5P

The M5P algorithm, which can create trees of regression models, is a construction of the M5 method. At the nodes, M5P mixes the linear regression and traditional decision tree algorithms.

Table 1. Datasets of structure

Metric	Definition
Coupling between Objects (CBO)	CBO is the count of other classes that a class is connected to, and vice versa.
Lack of Cohesion (LCOM)	By examining the instance variables or attributes used by methods, it gauges how differently different methods inside a class are from one another. Think of the class C1 as having n methods, M1, M2,..., Mn. let (I _i)= set of all instance variables used by method M _i . There are n such sets {I ₁ },.....{I _n }. Let P= {(I _i ,I _j) I I _i u I _j =0} and Q = {((I _i ,I _j) I I _i u I _j =0}. If all n sets { (I1 },.....(In)} are 0 then P=0, LCOM= IpI – I Q I, if I P I> I Q I = 0 otherwise
Number of Children (NOC)	The number of immediate subclasses of a class in a hierarchy is known as the NOC.
Depth of Inheritance (DIT)	The maximum number of steps from a class node to the tree's root, as determined by the number of ancestor classes, is the depth of a class within the inheritance hierarchy.
Weighted Methods per Class (WMC)	The WMC is a total of all the methods' complexities within a class. Consider a class K1 that has methods M1,... Mn defined within it. Let C1,.....Cn be the method complexity.

4. Performance Evaluation measures

The confusion matrix is often used to assess the effectiveness of binary prediction models (see Table 2). The regularly employed assessment measures provided in were used in this investigation. These metrics include ROC analysis, Accuracy, Precision, Recall, Specificity, and F measure [19].

Table 2. Confusion Matrix

Observed	Predicted	
	1.00 (defect- prone)	0.00 (Not Fault- Prone)
1.00 (defect- prone)	True Defect Prone (TDP)	False Not Defect Prone (FNDP)
0.00 (Not Fault- Prone)	False Defect Prone (FDP)	True Not Defect Prone (TNDP)

4.1 Accuracy

It is described as the number of classes divided by one accurately guessed the total number of classes

$$\text{Accuracy} = \frac{\text{TFP} + \text{TNFP}}{\text{TFP} + \text{FNFP} + \text{FFP} + \text{TNFP}}$$

4.2 Precession

The ratio of the number of classes accurately predicted as fault-prone to the number of classes anticipated as fault-prone is known as precision, also known as correctness.

$$\text{Precession} = \frac{\text{TFP}}{\text{TFP} + \text{FFP}}$$

4.3 Recall

Another name for recall is sensitivity. It is described as the proportion of classes that were accurately predicted to be fault prone to all classes that are truly fault prone.

$$\text{Recall} = \frac{\text{TFP}}{\text{TFP} + \text{FNFP}}$$

To assess how well projected fault proneness models work, precision and recall are crucial metrics. Better recall values imply that fewer fault-prone classes are unnoticed and higher accuracy values imply that less testing and inspection work is spent. Precision and recall metrics can be traded off. The precision will be one but the recall will be low if there are other classes that are fault-prone, for instance, if a suggested model predicts a class to be faulty and this class is indeed defective.

In a further illustration, if a fault-proneness model predicts that all classes are flawed, the recall will be one but the anticipated model's accuracy will be poor. As a result, the F-measure, a new measure that combines recall and accuracy, is required.

4.4 F Measure

The weighted harmonic mean of recall and accuracy is known as the F-measure. Typically, it is used to evaluate various ML algorithms by combining the Recall and Precision metrics into a single measure. The F-measure formula can be found in: It uses the harmonic mean of the accuracy and recall measurements.

$$\text{F Measure} = \frac{2 * \text{Precession} * \text{Recall}}{\text{Precession} + \text{Recall}}$$

4.5 Cross validation

Model accuracy should be predicted by applying it to various data sets. Therefore, we validated models using k-cross. The data set is partitioned into k subgroups at random. The other k-1 subsets are utilised to create a training set each time, and one of the k subsets is used as the test set. As a result, we determine how fault-prone each of the k classes is.

5. Experimental Results & Analysis

The ISTQB V 2.2 platform was utilised in this experiment to anticipate software problems using the eight machine learning techniques outlined above. The tested dataset was subjected to percentage split (55%, 65%, 75%, and 85%) and cross-validation (10-fold) testing modes. The statistical metrics to assess the classifiers' error rates in foretelling the software flaws in the chosen dataset are shown in Table 3.

The LR method and SMOReg algorithms both produced strong correlation coefficient (R2) values in the K1 dataset, as shown in Table II. The SMOReg method, on the other hand, had the lowest error rates for the remaining measures. As a consequence, out of the eight ML methods, the SMOReg classifier had the greatest performance results. The ANN algorithm, on the other hand, received the lowest performance ratings. Additionally, because the values were constant throughout all of the input data, the algorithm was unable to anticipate the legitimate values, resulting in error rates for both metrics, RS and RMSE, that exceeded 100%.

Table. 3 statistics on the ML algorithms' performance by using 10-fold cross-validation testing method

Classifier	AUC	F-Measures	Recall	Precession	Accuracy
ANN	0.1584	6.9869	14.7296	115.6751	134.899
Random Forest (RF)	0.4995	4.4208	9.3905	73.1899	86.0011
Random Tree (RT)	0.1317	5.5088	12.2059	91.2029	111.7856
Decision Table (DT)	0.2663	4.9232	11.2294	81.5082	102.8431
Linear regression (LR)	0.7594	6.5889	11.1759	109.0851	102.3526
Gaussian Processes (GP)	0.6794	4.6329	7.9437	76.7025	72.7514
SMOReg	0.7383	4.3159	7.4378	73.1085	68.1179
M5P	0.6989	4.3618	7.7834	72.2135	71.2835

Using the CFS approach mentioned in table1, the subset of characteristics was chosen. From the group of eight metrics, DIT, CBO, RFC, NPM, and LOC were chosen. Tables 3 display the accuracy, precision, recall, F-measure, and AUC measures' standard deviations (Std. Dev.) and percentages (%). To 274 classes, the anticipated model was applied. As a result, the ML algorithm -predicted model obtains excellent accuracy (74.24%), recall (79%) and F-measure (75%) as well as high AUC (0.81).

The percentage of accuracy and the accuracy measure's standard deviation are plotted in Figure 1. Similar to Figures 1, Figures 1 shows the value of additional measurements vs their standard deviation (precision, recall, and F-measure). The findings support the use of ML algorithm in building predictive models for defective courses. To verify the validity of the projected model, further comparable research needs to be conducted.

Table 4. Performance Measures through standard deviation

Method	%	Standard deviation
Accuracy	74.24	699
Precession	72	0.08
Recall	79	0.10
F-Measures	75	0.07
AUC	0.81	0.07

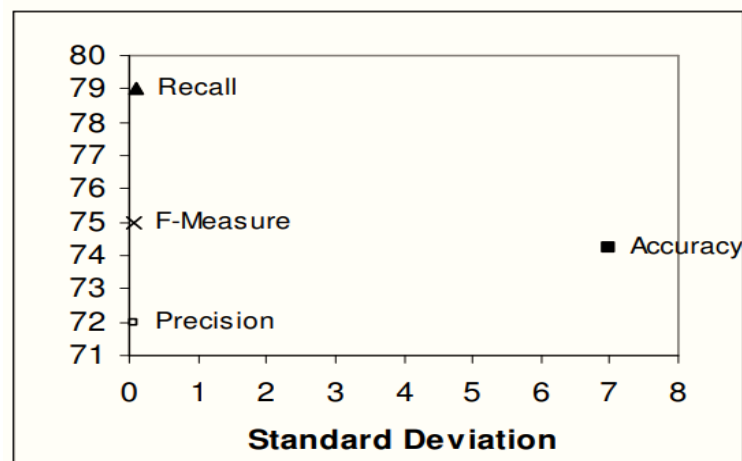


Fig 1. Performance measures vs standard deviation

Table 5. Results of statistical testing using % split mode for ML algorithms

Classifier	60%	70%	80%	90%
ANN	6.3554	5.097	5.7436	0.8393
Random Forest (RF)	3.9995	3.604	1.9424	0.2115
Random Tree (RT)	4.3654	4.2973	1.3014	0.0303
Decision Table (DT)	6.2844	5.5632	2.5614	2.5614
Linear regression (LR)	5.9445	5.5295	4.2484	2.6998
Gaussian Processes (GP)	5.1449	5.0989	3.4081	1.9759
SMOreg	4.434	3.9551	2.7383	1.1206
M5P	4.0811	5.9991	3.0258	0.5304

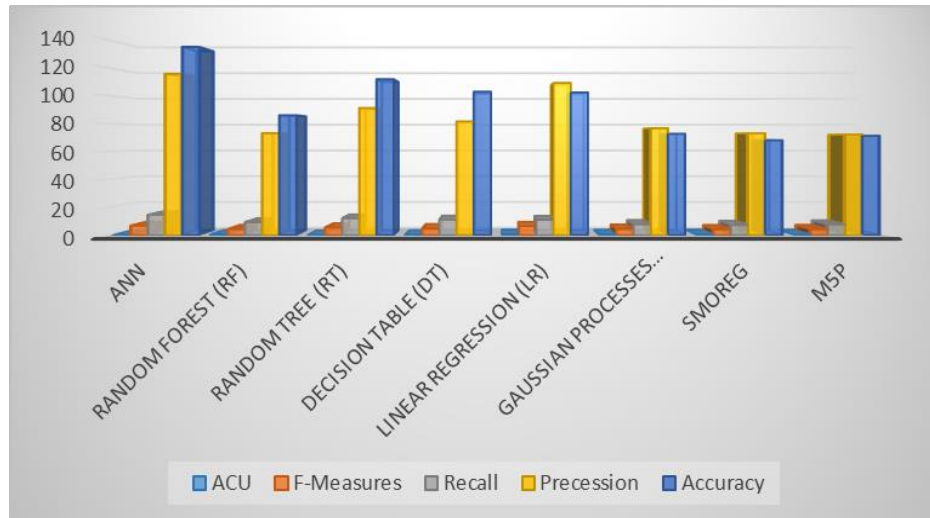


Fig. 2. Standard deviation Error in algorithms

6. Conclusion

Software must evolve, thus over time changes are made to the programme to accommodate shifting consumer and market demands. Numerous software flaws may occur during programme development. To discover these new flaws, the software developer should update the test suites by adding new test cases.

The ML Algorithm through standard deviation has an accuracy of 74.24 percent, a precision of 72%, a recall of 79%, an F-measure of 75%, and an AUC of 0.81. This study supports the feasibility, adaptability to OO systems, and utility of building ML models in forecasting error-prone classes. While research is ongoing, practitioners and academics can use ML to build a model to forecast incorrect classifications.

In the context of software testing, the prediction performance of ML, particularly in recall measure, can contribute to better software quality by lowering the likelihood that defective classes would go undetected. To provide conclusions that may be used across many businesses, future work may involve doing research of this type using diverse data sets. To develop a genetic algorithm-based prediction model, we want to duplicate our work. In order to assess the economic viability of a certain fault proneness model, we will also concentrate on cost-benefit analyses of various models.

ML-based prediction models may be used to forecast software defects. The programme can forecast upcoming software problems using past data. Using several ML techniques, this article assessed the accuracy of software fault prediction. We employed the ANN, RF, RT, DT, LR, GP, SMOreg, and M5P machine learning algorithms. A publicly accessible dataset was utilised to test the algorithms, and several statistical methods were employed to assess the error rates of the anticipated values.

Accuracy, Precision, Recall, F-Measures, and AUC are the metrics. We may infer from the findings that ML algorithms are effective methods for anticipating upcoming software faults. First, the findings were evaluated using a 10-fold cross-validation testing method. In contrast, the RT algorithm obtained the lowest F-measures error rate for both (79% and 85%) training sets while employing percentage split testing mode.

References

- [1] N. Seliya, T. M. Khoshgoftaar, and J. van Hulse, "Predicting faults in high assurance software," in *Proceedings of IEEE International Symposium on High Assurance Systems Engineering*, 2010, pp. 26–34. doi: 10.1109/HASE.2010.29.
- [2] S. Chatterjee and A. Roy, "Web software fault prediction under fuzzy environment using MODULO-M multivariate overlapping fuzzy clustering algorithm and newly proposed revised prediction algorithm," *Applied Soft Computing Journal*, vol. 22, pp. 372–396, 2014, doi: 10.1016/j.asoc.2014.03.030.
- [3] G. Abaci and A. Selamat, "A survey on software fault detection based on different prediction approaches," *Vietnam Journal of Computer Science*, vol. 1, no. 2, pp. 79–95, May 2014, doi: 10.1007/s40595-013-0008-z.
- [4] C. Catal, "A comparison of semi-supervised classification approaches for software defect prediction," *Journal of Intelligent Systems*, vol. 23, no. 1, pp. 75–82, Jan. 2014, doi: 10.1515/jisys-2013-0030.
- [5] Z. Li, X. Y. Jing, X. Zhu, H. Zhang, B. Xu, and S. Ying, "On the Multiple Sources and Privacy Preservation Issues for Heterogeneous Defect Prediction," *IEEE Transactions on Software Engineering*, vol. 45, no. 4, pp. 391–411, Apr. 2019, doi: 10.1109/TSE.2017.2780222.
- [6] J. Lu, V. Behbood, P. Hao, H. Zuo, S. Xue, and G. Zhang, "Transfer learning using computational intelligence: A survey," *Knowl Based Syst*, vol. 80, pp. 14–23, May 2015, doi: 10.1016/j.knosys.2015.01.010.
- [7] D. Sharma and P. Chandra, "Software fault prediction using machine-learning techniques," in *Smart Innovation, Systems and Technologies*, 2018, vol. 78, pp. 541–549. doi: 10.1007/978-981-10-5547-8_56.
- [8] R. Malhotra, "A systematic review of machine learning techniques for software fault prediction," *Applied Soft Computing Journal*, vol. 27, pp. 504–518, 2015, doi: 10.1016/j.asoc.2014.11.023.
- [9] A. Hammouri, M. Hammad, M. Alnabhan, and F. Alsarayrah, "Software Bug Prediction using machine learning approach," *International Journal of Advanced Computer Science and Applications*, vol. 9, no. 2, pp. 78–83, 2018, doi: 10.14569/IJACSA.2018.090212.
- [10] R. Malhotra, "Comparative analysis of statistical and machine learning methods for predicting faulty modules," *Applied Soft Computing Journal*, vol. 21, pp. 286–297, 2014, doi: 10.1016/j.asoc.2014.03.032.
- [11] S. K. Niranjana, Chirala Engineering College, IEEE Computational Intelligence Society, and Institute of Electrical and Electronics Engineers, *Proceedings of the 2017 International Conference on Big Data Analytics and Computational Intelligence: ICBDAI 2017: 23-25 March 2017, Chirala Engineering College, Chirala, Andhra Pradesh, India*.
- [12] D. L. Gupta and K. Saxena, "Software bug prediction using object-oriented metrics," *Sadhana - Academy Proceedings in Engineering Sciences*, vol. 42, no. 5, pp. 655–669, May 2017, doi: 10.1007/s12046-017-0629-5.
- [13] S. Kassaymeh, S. Abdullah, M. A. Al-Betar, and M. Alweshah, "Salp swarm optimizer for modeling the software fault prediction problem," *Journal of King Saud University* -

- Computer and Information Sciences*, vol. 34, no. 6, pp. 3365–3378, Jun. 2022, doi: 10.1016/j.jksuci.2021.01.015.
- [14] Anshu Parashar, Raman Kumar Goyal, Sakshi Kaushal, and Sudip Kumar Sahana, “Machine learning approach for software defect prediction using multi-core parallel computing,” *Automated Software Engineering*, vol. 29, no. 44, Jun. 2022.
 - [15] A. Hammouri, M. Hammad, M. Alnabhan, and F. Alsarayrah, “Software Bug Prediction using machine learning approach,” *International Journal of Advanced Computer Science and Applications*, vol. 9, no. 2, pp. 78–83, 2018, doi: 10.14569/IJACSA.2018.090212.
 - [16] A. Abdou, F. Akmel, and E. Birihanu, “A Literature Review Study of Software Defect Prediction using Machine Learning Techniques Related papers Early Prediction of Software Defect using Ensemble Learning: A Comparative Study A Literature Review Study of Software Defect Prediction using Machine Learning Techniques,” 2017. [Online]. Available: www.ermt.net
 - [17] Manjula. C. M. Prasad, L. F. Florence, and A. Arya3, “A Study on Software Metrics based Software Defect Prediction using Data Mining and Machine Learning Techniques,” *International Journal of Database Theory and Application*, vol. 8, no. 3, pp. 179–190, Jun. 2015, doi: 10.14257/ijdta.2015.8.3.15.
 - [18] A. Okutan and O. T. Yıldız, “Software defect prediction using Bayesian networks,” *Empir Softw Eng*, vol. 19, no. 1, pp. 154–181, 2014, doi: 10.1007/s10664-012-9218-8.
 - [19] D. Vasumathi, M. V. (2020). Unsupervised Learning Methods for Anomaly Detection and Log Quality Improvement Using Process Event Log. *International Journal of Advanced Science and Technology*, 29(1), 1109 - 1125. Retrieved from <http://sersc.org/journals/index.php/IJAST/article/view/3603>